

VentaGO 화면 깜빡임 코드 검토 및 개선안

Next.js Pages Router · React 18 · MUI 5
검토일 2026-08-05 | 정적 코드 검토

핵심 결론

깜빡임 가능성이 있는 코드가 존재한다. 가장 우선적인 원인은 모든 라우트 전환에서 현재 콘텐츠를 즉시 제거하고 고정 스켈레톤으로 교체하는 전역 처리다. 새로고침 시에는 저장된 테마와 사이드바 설정이 첫 페인트 이후 복원되어 색상 또는 폭이 한 번 바뀔 수 있다.

검토 범위

- ventago-app/src/pages/_app.tsx의 전역 라우트 전환 처리
- PageTransitionSkeleton의 렌더 구조
- SettingsProvider의 localStorage 복원 시점
- UserLayout 및 Layout의 반응형 레이아웃 교정 effect
- AuthGuard, GuestGuard, AdGuard의 스피너 및 리다이렉트 흐름

주의: 본 문서는 정적 코드 검토 결과다. 실제 발생 빈도와 프레임 지속시간은 브라우저 성능 기록으로 최종 확인해야 한다.

1. 발견 사항 요약

우선순위	발생 구간	예상 증상	판단
1	라우트 이동	오른쪽 콘텐츠가 스켈레톤으로 번쩍임	구조상 발생
2	새로고침/직접 진입	테마 색상·사이드바 폭이 한 번 바뀜	설정 차이 시 발생
3	모바일 진입/창 크기 변경	세로·가로 레이아웃 또는 메뉴 폭 이동	확인 필요
참고	인증 복원	전체 화면 스피너 노출	의도된 로딩이나 체감상 깜빡임 가능

2. 원인 A - 라우트 전환 시 콘텐츠 강제 교체

대상: ventago-app/src/pages/_app.tsx 250-294행, src/components/layout/PageTransitionSkeleton.tsx 9-24행

routeChangeStart 이벤트가 발생하면 isRouteChanging을 즉시 true로 만들고, 현재 페이지 컴포넌트를 제거한 뒤 고정 형태 스켈레톤을 렌더한다. 전환 완료 후 새 페이지를 다시 마운트한다. 따라서 빠른 라우트 이동에서도 최소 한 번의 시각적 교체가 생긴다.

```
const handleStart = () => setIsRouteChanging(true)
```

```
const innerContent = isRouteChanging  
? <PageTransitionSkeleton isLoading={true} />  
: <Component {...pageProps} />
```

왜 깜빡임처럼 보이는가

기존 화면과 스켈레톤의 구조, 높이, 배경 분포가 크게 다르다. 특히 400px 고정 사각형이 즉시 나타나므로 사용자는 로딩 피드백보다 화면이 한번 지워졌다고 인식할 수 있다. NProgress도 동시에 동작해 로딩 신호가 중복된다.

권장안 A1 - 지연 임계값 적용

전환 시작 직후에는 기존 콘텐츠를 유지하고, 150-200ms를 넘는 전환에서만 스켈레톤을 표시한다. 짧은 전환의 깜빡임을 제거하면서 느린 전환에는 피드백을 유지할 수 있다.

```
const timerRef = useRef<ReturnType<typeof setTimeout> | null>(null)  
  
const handleStart = () => {  
  timerRef.current = setTimeout(() => setIsRouteChanging(true), 180)  
}  
  
const handleEnd = () => {  
  if (timerRef.current) clearTimeout(timerRef.current)  
  setIsRouteChanging(false)  
}
```

중요: effect 정리 함수에서도 타이머를 반드시 해제해야 한다. routeChangeError에서도 handleEnd를 호출한다.

3. 라우트 전환 개선 선택지

선택지	장점	단점	권장도
A. NProgress만 사용	가장 단순하며 현재 콘텐츠 유지	느린 페이지에서 빈 로딩 상태 가능	높음
B. 180ms 후 스켈레톤	빠른 전환 깜빡임 제거	타이머 정리 필요	가장 높음
C. 기존 화면 위 오버레이	레이아웃 연속성 유지	클릭 차단·접근성 처리 필요	중간
D. 페이지별 스켈레톤	실제 화면과 형태 일치	구현·유지비가 큼	선별 적용

프로젝트 권장 조합

기본은 NProgress를 유지하고, 180ms를 초과하는 경우에만 콘텐츠 영역 스켈레톤을 표시한다. 판매 화면처럼 입력 연속성이 중요한 페이지는 이전 콘텐츠를 유지하는 방식을 우선한다.

4. 원인 B - 저장 설정의 첫 페인트 이후 복원

대상: ventago-app/src/@core/context/settingsContext.tsx 119-134행

SettingsProvider는 initialSettings로 먼저 렌더한 후 useEffect에서 localStorage를 읽어 저장 설정을 적용한다. 저장값이 기본값과 다르면 첫 화면과 두 번째 화면의 테마, 메뉴 접힘, 콘텐츠 폭이 달라진다.

```
const [settings, setSettings] = useState({ ...initialSettings })

useEffect(() => {
  const restored = restoreSettings()
  if (restored) setSettings(restored)
}, [pageSettings])
```

권장안 B1 - 클라이언트 초기 상태 함수

브라우저에서만 렌더되는 전제라면 lazy initializer로 저장값을 초기 상태에 반영한다. 다만 SSR HTML과 클라이언트 결과가 달라질 수 있으므로 hydration 경고 여부를 반드시 확인해야 한다.

```
const [settings, setSettings] = useState<Settings>(() => {
  if (typeof window === 'undefined') return { ...initialSettings }
  return restoreSettings() ?? { ...initialSettings }
})
```

권장안 B2 - 쿠키 기반 SSR 동기화

가장 안정적인 방식은 테마·레이아웃 핵심값을 쿠키에 저장하고 서버 렌더 단계에서 초기 설정으로 전달하는 것이다. 서버와 클라이언트가 같은 값으로 첫 화면을 그리므로 테마 플래시를 방지할 수 있다.

5. 원인 C - 반응형 레이아웃의 사후 교정

대상: ventago-app/src/@core/layouts/Layout.tsx 18-36행, src/layouts/UserLayout.tsx 139-146행

hidden 값이 바뀐 뒤 두 컴포넌트의 effect가 각각 레이아웃 설정을 저장한다. 기존 설정으로 한 번 렌더한 다음 vertical 및 navCollapsed 값을 적용하므로 모바일 첫 진입이나 창 크기 변경 때 폭 이동이 보일 수 있다. 두 effect가 오래된 settings 객체를 기반으로 연속 저장할 가능성도 있다.

권장 수정 방향

- 렌더에 사용할 effectiveLayout을 hidden ? 'vertical' : settings.layout 방식으로 즉시 계산한다.
- 반응형 교정과 localStorage 저장 책임을 한 컴포넌트로 통합한다.
- 여러 필드를 갱신할 때는 최신 상태를 기준으로 병합할 수 있도록 saveSettings API를 함수형 업데이트 형태로 확장한다.
- useMediaQuery의 SSR 기본값과 실제 viewport가 다른 경우 첫 페인트가 달라지는지 모바일에서 확인한다.

```
const effectiveLayout = hidden ? 'vertical' : settings.layout

return effectiveLayout === 'horizontal'
  ? <HorizontalLayout {...props}>>{children}</HorizontalLayout>
  : <VerticalLayout {...props}>>{children}</VerticalLayout>
```

6. 인증 화면의 스피너 전환

AuthGuard는 auth.loading 또는 user가 null인 동안 전체 fallback Spinner를 렌더한다. 새로고침 시 /auth/me 응답 전까지 나타나는 의도된 동작이지만, 응답이 매우 빠르면 짧은 전체 화면 스피너가 깜빡임처럼 보일 수 있다.

판단

인증 검증 전 보호 화면을 노출해서는 안 되므로 스피너 제거를 우선 권장하지 않는다. 대신 앱 배경과 동일한 정적 셀을 유지하거나 최소 표시 시간 없이 지연 표시하는 방법을 검토한다. 보안을 해치는 선렌더는 금지한다.

7. 적용 순서

1	전역 라우트 스켈레톤에 180ms 임계값 적용 또는 제거
2	새로고침 시 settings 복원 전후의 테마·메뉴 폭 측정
3	반응형 레이아웃 교정 effect를 단일 책임으로 통합
4	인증 복원 스피너는 보안 유지 조건에서 시각적 셀만 개선
5	Chrome Performance와 Slow 3G 조건으로 회귀 검증

8. 검증 체크리스트

- 01. 사이드바 메뉴를 빠르게 연속 클릭해 콘텐츠가 빈 화면 또는 고정 스켈레톤으로 교체되지 않는가
- 02. 라우트 전환이 180ms 미만일 때 스켈레톤이 나타나지 않는가
- 03. 느린 네트워크에서 NProgress 또는 스켈레톤 중 의도한 피드백이 유지되는가
- 04. 다크 모드 저장 후 새로고침해 밝은 배경이 순간적으로 보이지 않는가
- 05. 사이드바 접힘 상태 저장 후 새로고침해 폭이 왕복하지 않는가
- 06. 모바일 첫 진입에서 horizontal 레이아웃이 잠깐 나타나지 않는가
- 07. 창 너비를 lg breakpoint 전후로 반복 변경해 메뉴 폭이 두 번 움직이지 않는가
- 08. 로그인 상태 복원 중 보호 페이지 콘텐츠가 먼저 노출되지 않는가
- 09. routeChangeError 발생 후 로딩 상태와 타이머가 정상 해제되는가
- 10. React Strict Mode 개발 환경과 프로덕션 빌드 모두에서 결과가 같은가

권장 계측

- Chrome DevTools Performance에서 routeChangeStart부터 새 페이지 paint까지 기록
- Network Fast 3G 및 Slow 3G 조건 비교
- Layout Shift 영역 표시로 사이드바·콘텐츠 폭 변화 확인
- 최소 10회 전환 후 증상 재현 비율과 최대 공백 시간을 기록

완료 기준

빠른 라우트 전환에서 콘텐츠 교체 플래시가 없고, 새로고침 시 테마·사이드바가 첫 페인트부터 최종 상태와 일치하며, 인증 검증 중 보호 콘텐츠가 노출되지 않아야 한다.